TEMA: Generación de relaciones entre tablas

Resultados de Aprendizaje:

- Que los estudiantes pueden crear relaciones entre tablas de tipo uno a muchos

DESARROLLO

Cuando tabajamos con los modelos para la generacion de la base de datos utilizamos una gran variedad de tipos , pero Django ya ha pensado en la mayoria de esos tipos de datos que usted podria necesitat algo de ellos son los sigueintes

Tipo de campo Django	Tipo de dato	Explicación	Ejemplo
CharField	Texto corto	Guarda cadenas de longitud limitada. Normalmente requiere max_length. Ideal para nombres, títulos, códigos, etc.	nombre = models.CharField(max_length=100)
TextField	Texto largo	Guarda textos extensos. Ideal para descripciones, comentarios o contenido.	descripcion = models.TextField()
IntegerField	Entero	Guarda números enteros positivos o negativos.	cantidad = models.IntegerField()
PositiveIntegerField	Entero positivo	Guarda números enteros mayores o iguales a 0. Muy utilizado para stock o cantidades.	stock = models.PositiveIntegerField()

Tipo de campo Django	Tipo de dato	Explicación	Ejemplo
PositiveSmallIntegerField	Entero positivo pequeño	Similar a PositiveIntegerField, pero pensado para valores pequeños.	nivel = models.PositiveSmallIntegerField()
SmallIntegerField	Entero pequeño	Guarda enteros positivos o negativos dentro de un rango menor.	nivel = models.SmallIntegerField()
BigIntegerField	Entero grande	Permite almacenar números enteros mucho más grandes que IntegerField.	codigo = models.BigIntegerField()
FloatField	Decimal aproximado	Guarda números con decimales usando punto flotante. Puede tener errores de precisión.	temperatura = models.FloatField()
DecimalField	Decimal exacto	Guarda números decimales con precisión definida. **Recomendado para dinero y precios.**	precio = models.DecimalField(max_digits=10, decimal_places=2)
BooleanField	Booleano	Guarda True o False.	activo = models.BooleanField(default=True)
DateField	Fecha	Guarda solamente una fecha: año, mes y día.	fecha = models.DateField()
TimeField	Hora	Guarda solamente una hora.	hora = models.TimeField()
DateTimeField	Fecha y hora	Guarda fecha y hora. Puede utilizar auto_now o auto_now_add.	created_at = models.DateTimeField(auto_now_add=True)

Tipo de campo Django	Tipo de dato	Explicación	Ejemplo
DurationField	Duración	Guarda una cantidad de tiempo o duración. En Python se maneja como timedelta.	duracion = models.DurationField()
EmailField	Correo electrónico	Es un CharField que además valida que el valor tenga formato de correo electrónico.	correo = models.EmailField()
URLField	URL	Guarda texto y valida que tenga formato de URL.	sitio = models.URLField()
SlugField	Slug	Guarda texto adecuado para URLs, como computadoras-portatiles.	slug = models.SlugField()
UUIDField	UUID	Guarda identificadores UUID. Útil cuando no quieres utilizar IDs enteros tradicionales.	codigo = models.UUIDField()
BinaryField	Binario	Guarda datos binarios (bytes). No suele utilizarse para archivos grandes.	datos = models.BinaryField()
JSONField	JSON	Permite almacenar estructuras JSON como diccionarios, listas, números, etc.	configuracion = models.JSONField()

Tipo de campo Django	Tipo de dato	Explicación	Ejemplo
FileField	Archivo	Guarda la **ruta/referencia de un archivo** administrado mediante el sistema de almacenamiento de Django.	archivo = models.FileField(upload_to="archivos/")
ImageField	Imagen	Especialización de FileField para imágenes. Normalmente requiere **Pillow**.	imagen = models.ImageField(upload_to="productos/")
AutoField	Entero autoincremental	Campo entero que se incrementa automáticamente. Tradicionalmente utilizado para claves primarias.	id = models.AutoField(primary_key=True)
BigAutoField	Entero grande autoincremental	Igual que AutoField, pero admite un rango mucho mayor. Es el tipo de PK automático habitual en proyectos Django actuales.	id = models.BigAutoField(primary_key=True)
ForeignKey	Relación N:1	Crea una relación de **muchos a uno**. Por ejemplo, muchos productos pueden pertenecer a una categoría.	categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE)
OneToOneField	Relación 1:1	Relaciona exactamente un registro con otro registro.	perfil = models.OneToOneField(Perfil, on_delete=models.CASCADE)

Tipo de campo Django	Tipo de dato	Explicación	Ejemplo
ManyToManyField	Relación N:M	Permite que múltiples registros se relacionen con múltiples registros. Django normalmente crea una tabla intermedia.	categorias = models.ManyToManyField(Categoria)

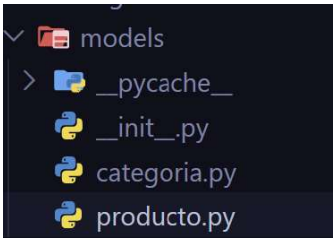
Requerimientos:

Para la realizacion de esta guia usted debe de contar con los siguientes elementos

- Contar con el proyecto comercial generado en la guia anterios
- Contar con un servidor de base de datos mysql

Relaciones entre tablas

1. Para esta guia a dado que crearemos el modelo de producto y este tiene asociada una imagen necesitaremo instalar una librería mas llama pillow esto lo aremos con el comando **> pip install pillow**
2. Aprande a realcioar tablas en Djanog es fundamenteal para poder manejar datos interconectados
3. Para poder realizar nuestra pimera relacion crearemos un nuevo aarchivo **> models/producto.py**



4. Hoy agregaremos el codigo necesario para poder agergar la tabla de productos que definimos en la guia anterior

```python
from django.db import models
from .categoria import Categoria


class Producto(models.Model):
    nombre = models.CharField(max_length=100, verbose_name="Nombre")
    descripcion = models.TextField(
        max_length=255, null=True, blank=True, verbose_name="Descripción")
    # max_digits es el número máximo de dígitos que puede tener el número, incluyendo los decimales
    precio_compra = models.DecimalField(
        max_digits=10, decimal_places=2, verbose_name="Precio de compra")
    precio_venta = models.DecimalField(
        max_digits=10, decimal_places=2, verbose_name="Precio de venta")
    stock = models.PositiveIntegerField(verbose_name="Stock")
    # también lo puede ponder de esta manera
    # stock = models.IntegerField()B
    # relación de uno a muchos, un producto pertenece a una categoria y una categoria puede tener muchos productos
    categoria = models.ForeignKey(
        Categoria, on_delete=models.CASCADE, related_name='productos', verbose_name="Categoría")
    # la imagen se guardará en la carpeta productos dentro de media, null=True permite que el campo sea nulo y
    blank=True permite que el campo sea opcional
    imagen = models.ImageField(
        upload_to='productos/', null=True, blank=True, verbose_name="Imagen")
    # auto_now_add agrega la fecha de manera automática cuando se crea el objeto
    created_at = models.DateTimeField(
        auto_now_add=True, verbose_name="Fecha de creación")
    # auto_now actualiza la fecha de manera automática cuando se actualiza el objeto
    updated_at = models.DateTimeField(
        auto_now=True, verbose_name="Fecha de actualización")

    def __str__(self):
        return f"id: {self.id} Nombre: {self.nombre} Precio de compra: {self.precio_compra} Precio de venta:
        {self.precio_venta} Stock: {self.stock}"
```

Explicaion del codigo

- Improtamos la clase categoria que es la clase que esta relacionad con la clase productos

```python
from categoria import Categoria
```

- auto_now_add : agrega la fecha de manera automática cuando se crea el objeto
- auto_now : ctualiza la fecha de manera automática cuando se actualiza el objeto
- Agregamos la realcion de uno a muchos , en cuando al comportamiento de esta relacione tenemos CASCADE , que si elimina al padre todos los datos hijos seran eliminado tambien , DO_NOTHING no hacer nada si eliminar a la clase padre , PROTECT que actuara enviando un menaje cuando intente elimnar una catgoria que tenga asociado un producto , de modo que impedira su eliminacion hasta que no tenga dependencias

```python
categoria = models.ForeignKey(Categoria,
on_delete=models.CASCADE, related_name='productos')
```

5. El siguiente paso es modificar el archivo ❯ **catalogo/models/__init__.py** , y agregar la importación del modelo que se transforma en la tabla en la base de datos , **si creamos más modelos todo deberán ser importados a estos archivos**
6. Quedando de la siguiente manera

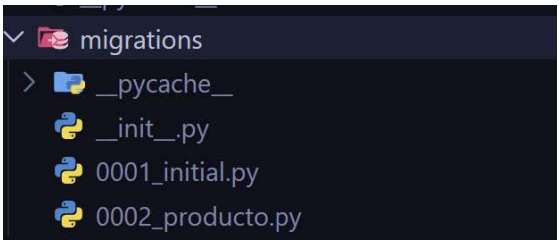
```python
from .categoria import Categoria
from .producto import Producto
```

Generacion de migraciones

7. El siguiente paso de deberemos de realizar es la genracion de las migraciones esto lo realizaremos con el comando ❯ **python .\manage.py makemigration**

```
❯ python .\manage.py makemigrations
```

8. Agragando una migracion ma a nuesto proyecto



9. si abrimo la maigracion 0002 podremos visualizar el codigo de esta

```python
import django.db.models.deletion
from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ('catalogo', '0001_initial'),
    ]

    operations = [
        migrations.CreateModel(
            name='Producto',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('nombre', models.CharField(max_length=100)),
                ('descripcion', models.TextField(max_length=255, null=True)),
                ('precio_compra', models.DecimalField(decimal_places=2, max_digits=10)),
                ('precio_venta', models.DecimalField(decimal_places=2, max_digits=10)),
                ('stock', models.PositiveIntegerField()),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
                ('categoria', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE, related_name='productos'
                to='catalogo.categoria')),
            ],
        ),
    ]
```

10. para terminar de crear la tabla debera de ejecutar el comando de las migraciones ❯ **python .\manage.py migrate**
11. si todo a salido bien le se mostrar un mensaje como el siguiente

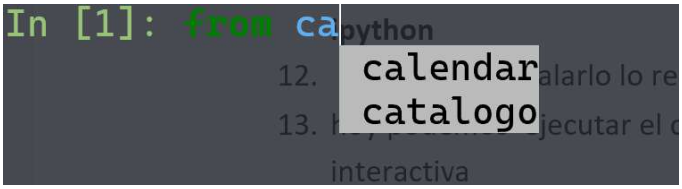
```
❯ python .\manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, catalogo, contenttypes, sessions
Running migrations:
  Applying catalogo.0002_producto... OK
```

Terminal interactiva

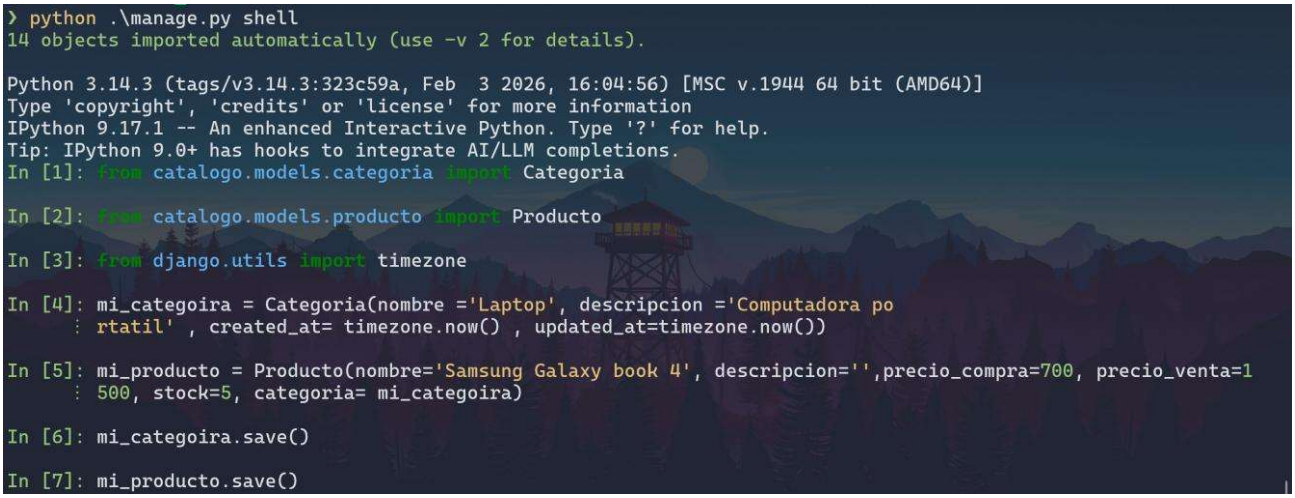
12. en la seacion anterior hemos trabajado desde la terminal par poder interactuar con la base de datos , pero teniemos problemas porque no se autocompetaba nada ni podemos utilziar tab para

mostrar mas elemtos , para poder lograr algo mas interactivo podemos agergar el paquete **ipython**

13. para poder instalarlo lo realizaremos con el comando siguiente **> pip install ipython**
14. hoy podemos ejecutar el comanod **❯ python .\manage.py Shell ,** que proporciona una consola interactiva
15. de modo que se puede comenzar a escribir código y al presionar Tab , presentara la ayuda de auto completado



16. de modo que tendremos que realizar la simportaciones necearias y primero crearemos una categora para poder asociar con el produto esto lo aremos de la siguiene manera



17. con lo anterior ya ha aprendido a como relacionar una tabla con otra y como poder agregar datos relacionados en la base de datos

	**EJERCICIOS DE PRÁCTICA:**

Cree las tablas faltantes del sigarma que comenzamos en las cesiones anteriroes

SISTEMA COMERCIAL

